



K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

Database Management System

Unit : 2 (SQL COMMANDS)

Assignment: (ADVANCED SQL COMMANDS)

Submitted by:

Deepak

2501730097

B.TECH CSE(AI & ML)

Section: A

Submitted to:

Ms. MansiKajal

g Part: 1 (Subqueries):

Question 1: An online store keeps two tables: CUSTOMER(Cust_ID, Name, City, Join_Date) and ORDERS(Order_ID, Cust_ID, Order_Date, Amount). Write SQL queries using subqueries to answer the following, and state for each whether your subquery is correlated or non-correlated:

(a) List the customers who have placed at least one order whose amount is greater than the average order amount of the whole store.

```
37 # 1.a)
38 • SELECT DISTINCT C.Cust_ID, C.Name, C.City
39 FROM CUSTOMER C
40 WHERE C.Cust_ID IN
41 (
42     SELECT O.Cust_ID
43     FROM ORDERS O
44     WHERE O.Amount > (SELECT AVG(Amount) FROM ORDERS)
45 );
```

Result Grid

Cust_ID	Name	City
1	Ajay	Jaipur
5	Shalu	Kotkapura
NULL	NULL	NULL

(b) List the customers who have never placed an order (write it once with NOT IN and once with NOT EXISTS, and explain why NOT EXISTS is safer when Cust_ID in ORDERS can be NULL).

```

47 #1.b)
48 • SELECT Cust_ID, Name, City
49 FROM CUSTOMER
50 WHERE Cust_ID NOT IN
51 (
52     SELECT Cust_ID
53     FROM ORDERS
54 );
55
56 #1.c)

```

Result Grid

Cust_ID	Name	City
NULL	NULL	NULL

*Since there was no customer who has never placed order so came empty.

NOT IN can give unexpected results if the subquery contains NULL, because comparisons with NULL result in UNKNOWN.

(c) For each city, find the customer whose total order value is the highest in that city (hint: derived table + multi-column comparison or a correlated subquery).

Question 2: A hospital database has DOCTOR(Doc_ID, Doc_Name, Specialization) and PATIENT(Pat_ID, Pat_Name, Doc_ID, Admit_Date, Bill_Amount).

```

56 #1.c)
57 • SELECT C.Cust_ID, C.Name, C.City, T.Total_Amount
58 FROM CUSTOMER C
59 JOIN
60 (
61     SELECT Cust_ID, SUM(Amount) AS Total_Amount
62     FROM ORDERS
63     GROUP BY Cust_ID
64 ) T
65 ON C.Cust_ID = T.Cust_ID
66 WHERE (C.City, T.Total_Amount) IN
67 (
68     SELECT C2.City, MAX(T2.Total_Amount)
69     FROM CUSTOMER C2

```

Result Grid

Cust_ID	Name	City	Total_Amount
1	Ajay	Jaipur	5000.00
2	Bhanu	Noida	3500.00
3	Vikas	Ajmer	4200.00
4	Shanu	Delhi	2800.00

(a) Using a correlated subquery, list patients whose bill is higher than the average bill of all patients treated by the same doctor.

```

117 #2.a)
118 • SELECT P.Pat_ID, P.Pat_Name, P.Bill_Amount
119 FROM PATIENT P
120 WHERE P.Bill_Amount >
121 (
122     SELECT AVG(P2.Bill_Amount)
123     FROM PATIENT P2
124     WHERE P2.Doc_ID = P.Doc_ID
125 );
126
Result Grid | Filter Rows: | Edit:


| Pat_ID | Pat_Name | Bill_Amount |
|--------|----------|-------------|
| 102    | Bhanu    | 18000.00    |
| 103    | Vikas    | 12000.00    |
| 106    | Ravi     | 25000.00    |
| 108    | Pooja    | 35000.00    |
| 110    | Karan    | 18000.00    |


PATIENT 4 x

```

(b) Using ALL, list the doctors for whom every patient has a bill above 10000. Then rewrite the same query using NOT EXISTS.

```

127 #2.b)
128 • SELECT D.Doc_ID, D.Doc_Name
129 FROM DOCTOR D
130 WHERE 10000 < ALL
131 (
132     SELECT P.Bill_Amount
133     FROM PATIENT P
134     WHERE P.Doc_ID = D.Doc_ID
135 );
136
137 #2.c)
138 • SELECT Y.Specialization, Y.Doc_ID
139 FROM DOCTOR Y
140 WHERE NOT EXISTS (
141     SELECT *
142     FROM PATIENT P
143     WHERE P.Doc_ID = Y.Doc_ID
144     AND P.Bill_Amount < 10000
145 );
146
Result Grid | Filter Rows: | Edit:


| Doc_ID | Doc_Name   |
|--------|------------|
| 1      | Dr. Sharma |
| 3      | Dr. Mehta  |
| 4      | Dr. Singh  |
| 5      | Dr. Gupta  |
| NULL   | NULL       |


DOCTOR 5 x

```

```

136 • SELECT D.Doc_ID, D.Doc_Name
137 FROM DOCTOR D
138 WHERE NOT EXISTS
139 (
140     SELECT 1
141     FROM PATIENT P
142     WHERE P.Doc_ID = D.Doc_ID
143           AND P.Bill_Amount <= 100000
144 );
145

```

Doc_ID	Doc_Name
1	Dr. Sharma
3	Dr. Mehta
4	Dr. Singh
5	Dr. Gupta
NULL	NULL

*In both the cases got same output only as difference was just of subquery used.

c) Using a subquery in the FROM clause, show each specialization with the number of doctors and the total bill amount collected, only for specializations whose total is more than 100000.

```

146 #2.c)
147 • SELECT X.Specialization, X.Doctors, X.Total_Bill
148 FROM
149 (
150     SELECT D.Specialization,
151            COUNT(DISTINCT D.Doc_ID) AS Doctors,
152            SUM(P.Bill_Amount) AS Total_Bill
153     FROM DOCTOR D
154     JOIN PATIENT P ON D.Doc_ID = P.Doc_ID
155     GROUP BY D.Specialization
156 ) X
157 WHERE X.Total_Bill > 100000;
158

```

Specialization	Doctors	Total_Bill
----------------	---------	------------

*Since there was no such case so in output came null.
(d) Explain in two or three sentences how the database evaluates the query in part (a) for a table of 1,000 patients.

For each patient, the inner query finds the average bill of patients treated by the same doctor. This average is then compared with the

current patient's bill, and only patients whose bill is higher than that average are selected. For 1,000 patients, this process is logically repeated for each patient, although the database may optimize it internally.

Part:2(SET Operations):

Question 1: A university maintains two club membership tables:

SPORTS_CLUB(Roll_No, Student_Name, Year) and MUSIC_CLUB(Roll_No, Student_Name, Year). Three students are members of both clubs.

(a) Write queries to list students who are in at least one club, students who are in both clubs, students who are only in the sports club, and students who are in exactly one club (hint: combine two EXCEPT results with UNION).

```
192 #1.a)
193 #At least one:
194 • SELECT Roll_No, Student_Name, Year
195 FROM SPORTS_CLUB
196 UNION
197 SELECT Roll_No, Student_Name, Year
198 FROM MUSIC_CLUB;
199
200 #Both clubs:
```

Result Grid			Filter Rows:		Export:
	Roll_No	Student_Name	Year		
▶	1	Ajay	1		
	2	Bhanu	2		
	3	Vikas	1		
	4	Shanu	3		
	5	Ravi	2		

```

200 #Both club:
201 • SELECT Roll_No, Student_Name, Year
202 FROM SPORTS_CLUB
203 INTERSECT
204 ✖ SELECT Roll_No, Student_Name, Year
205 FROM MUSIC_CLUB;
206

```

Result Grid | Filter Rows: | Export:

Roll_No	Student_Name	Year
2	Bhanu	2
3	Vikas	1
5	Ravi	2

```

207 #Only in sports:
208 • SELECT Roll_No, Student_Name, Year
209 FROM SPORTS_CLUB
210 ✖ EXCEPT
211 SELECT Roll_No, Student_Name, Year
212 FROM MUSIC_CLUB;
213
214 #Exactly one:
215 • (SELECT Roll_No, Student_Name, Year

```

Result Grid | Filter Rows: | Export:

Roll_No	Student_Name	Year
1	Ajay	1
4	Shanu	3
6	Karan	1

```

205 #Exactly one:
206 • (SELECT Roll_No, Student_Name, Year
207 FROM SPORTS_CLUB
208 ✖ EXCEPT
209 SELECT Roll_No, Student_Name, Year
210 FROM MUSIC_CLUB)
211
212 UNION
213
214 (SELECT Roll_No, Student_Name, Year
215 FROM MUSIC_CLUB
216 EXCEPT

```

Result Grid | Filter Rows: | Export:

Roll_No	Student_Name	Year
1	Ajay	1
4	Shanu	3
6	Karan	1
7	Aman	3
8	Neha	1

(b) If SPORTS_CLUB has 40 rows and MUSIC_CLUB has 25 rows, how many rows will UNION return and how many will UNION ALL return? Explain.

- UNION: 62 rows

- UNION ALL: 65 rows

Explanation:

There are 40 rows in SPORTS_CLUB and 25 rows in MUSIC_CLUB, so UNION ALL returns all $40 + 25 = 65$ rows. Since 3 students are members of both clubs, UNION removes those 3 duplicate rows:

$65 - 3 = 62$ rows.

(c) A student tried `SELECT Roll_No, Student_Name FROM SPORTS_CLUB UNION SELECT Roll_No FROM MUSIC_CLUB` and got an error. Explain the error and correct the query.

```

122 #1.c)
123 • SELECT Roll_No, Student_Name
124 FROM SPORTS_CLUB
125 UNION
126 SELECT Roll_No, Student_Name
127 FROM MUSIC_CLUB;

```

Roll_No	Student_Name
1	Ajay
2	Bhanu
3	Vikas
4	Shanu
5	Ravi

*The error occurs because both queries in a UNION must have the same number of columns. The first query returns two columns (Roll_No and Student_Name), while the second returns only one column (Roll_No). To correct it, both queries should return Roll_No and Student_Name in the same order.

Question 2: Two companies are merging. Their staff tables are STAFF_A(Emp_Code, Full_Name, Email, Dept) and STAFF_B(Emp_Code, Full_Name, Email, Dept). Emp_Codes are different in the two companies, but a person may exist in both tables with the same e-mail address.

(a) Write a query that lists e-mail addresses that appear in both companies (possible duplicate people).

```

260 #2.a)
261 • SELECT Email
262 FROM STAFF_A
263 INTERSECT
264 SELECT Email
265 FROM STAFF_B;

```

Result Grid | Filter Rows:

Email
ajay@gmail.com
vikas@gmail.com
neha@gmail.com

(b) Write a query that produces one combined staff directory (Full_Name, Email, Dept) without duplicates, sorted by Full_Name, and explain where the ORDER BY clause must be placed and why.

```

267 #2.b)
268 • SELECT Full_Name, Email, Dept
269 FROM STAFF_A
270 UNION
271 SELECT Full_Name, Email, Dept
272 FROM STAFF_B
273 ORDER BY Full_Name;

```

Result Grid | Filter Rows: | Export:

Full_Name	Email	Dept
Ajay Sharma	ajay@gmail.com	IT
Bhanu Singh	bhanu@gmail.com	HR
Karan Verma	karan@gmail.com	HR
Neha Gupta	neha@gmail.com	IT
Pooja Singh	pooja@gmail.com	Marketing

*ORDER BY should be placed at the end of the complete UNION query because UNION first combines the results from both tables and removes duplicates. Then ORDER BY Full_Name sorts the final combined result, rather than sorting each table separately.

(c) Write a query listing employees of company B whose e-mail does not exist in company A, first with EXCEPT and then with

NOT EXISTS (for a database that does not support EXCEPT).

```

276 #EXCEPT:
277 • SELECT Email
278 FROM STAFF_B
279 ✖ EXCEPT
280 SELECT Email
281 FROM STAFF_A;
282 #NOT EXISTS:
283 • SELECT B.Full_Name, B.Email, B.Dept
284 FROM STAFF_B B
285 WHERE NOT EXISTS
286 (
287 SELECT 1
288 FROM STAFF_A A
289 WHERE A.Email = B.Email

```

Result Grid | Filter Rows: | Export

Full_Name	Email	Dept
Karan Verma	karan@gmail.com	HR
Pooja Singh	pooja@gmail.com	Marketing

(d) Add a column named Source to the combined directory so that each row shows 'A' or 'B'. Which operator, UNION or UNION ALL, is correct now, and why?

* UNION ALL is correct because the Source column makes rows from company

```

291
292 #2.d)
293 • SELECT Full_Name, Email, Dept, 'A' AS Source
294 FROM STAFF_A
295 UNION ALL
296 SELECT Full_Name, Email, Dept, 'B' AS Source
297 FROM STAFF_B;
298

```

A and company B different ('A' vs 'B'). Even if the employee details are identical, the rows are no longer duplicates, so we want to keep both rows.

Result Grid | Filter Rows: | Export: | Wrap C

Full_Name	Email	Dept	Source
Bhanu Singh	bhanu@gmail.com	HR	A
Vikas Kumar	vikas@gmail.com	Finance	A
Ravi Mehta	ravi@gmail.com	Sales	A
Neha Gupta	neha@gmail.com	IT	A
Ajay Sharma	ajay@gmail.com	IT	B
Karan Verma	karan@gmail.com	HR	B
Vikas Kumar	vikas@gmail.com	Finance	B
Pooja Singh	pooja@gmail.com	Marketing	B
Neha Gupta	neha@gmail.com	IT	B

Part: 3(VIEW):

Question 1: A bank stores accounts in ACCOUNT(Acc_No, Cust_Name, Branch_Code, Balance, PIN, Aadhaar_No, Opened_On). Branch managers must see accounts of their own branch only, and must never see PIN or Aadhaar_No.

(a) Write the CREATE VIEW statement for the Mumbai branch (Branch_Code = 'MUM') that satisfies both requirements, and the GRANT statement that gives the manager access to the view only.

```
321 #1.a)
322 • CREATE VIEW mumbai_accounts AS
323 SELECT Acc_No, Cust_Name, Branch_Code, Balance, Opened_On
324 FROM ACCOUNT
325 WHERE Branch_Code = 'MUM'
326 WITH CHECK OPTION;
327 • SELECT * FROM mumbai_accounts;
328
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Acc_No	Cust_Name	Branch_Code	Balance	Opened_On
▶	1001	Ajay	MUM	50000.00	2020-01-15
	1002	Bhanu	MUM	75000.00	2021-03-20
	1004	Shanu	MUM	60000.00	2022-08-05

(b) Write a second view, BRANCH_TOTALS, showing each branch with the number of accounts and total balance.

(c) State which of the two views is updatable and why the other one is not.

mumbai_accounts is updatable because it is based on a single table and does not use GROUP BY or aggregate functions.

```
330 #1.b)
331 • CREATE VIEW BRANCH_TOTALS AS
332 SELECT Branch_Code,
333 COUNT(*) AS Number_Of_Accounts,
334 SUM(Balance) AS Total_Balance
335 FROM ACCOUNT
336 GROUP BY Branch_Code;
337 • SELECT * FROM BRANCH_TOTALS;
338
```

Result Grid | Filter Rows: | Export: |

	Branch_Code	Number_Of_Accounts	Total_Balance
▶	MUM	3	185000.00
	DEL	1	45000.00
	BLR	1	90000.00

BRANCH_TOTALS is not updatable because it uses GROUP BY, COUNT() and SUM(). It contains summarized data rather than individual account rows.

(d) A manager ran UPDATE mumbai_accounts SET Branch_Code = 'DEL' WHERE Acc_No = 1001 and the account disappeared from his view. What clause should have been added to the view to prevent this, and what would happen with that clause in place?

The view should have the **WITH CHECK OPTION** clause.

WITH CHECK OPTION;

It ensures that any row updated through the view must still satisfy the view's condition (Branch_Code = 'MUM').

So, if the manager tries to change the branch from MUM to DEL, the update will be rejected and the account will remain in the Mumbai view.

Question 2: An e-commerce reporting team runs the same query every morning: a join of PRODUCT(Prod_ID, Prod_Name, Category, Price), ORDER_ITEM(Order_ID, Prod_ID, Qty) and ORDERS(Order_ID, Order_Date, Cust_ID) that calculates product-wise total quantity and revenue for the previous day. The query takes 12 minutes on 200 million rows.

(a) Write a view named PRODUCT_SALES that returns Prod_Name, Category, Total_Qty and Total_Revenue.

391	SELECT
392	P.Prod_Name,
393	P.Category,
394	SUM(OI.Qty) AS Total_Qty,
395	SUM(OI.Qty * P.Price) AS Total_Revenue
396	FROM PRODUCT P
397	JOIN ORDER_ITEM OI
398	ON P.Prod_ID = OI.Prod_ID
399	JOIN ORDERS3 O
400	ON OI.Order_ID = O.Order_ID

<				
Result Grid			Filter Rows:	Export:
	Prod_Name	Category	Total_Qty	Total_Revenue
▶	Laptop	Electronics	3	150000.00
	Mouse	Electronics	8	8000.00
	Chair	Furniture	2	10000.00
	Keyboard	Electronics	2	4000.00

(b) Would a simple view make the morning query faster? Explain what a simple view actually does at execution time.

No, a simple view will not necessarily make the morning query faster. A simple view does not store the result of the query; it stores only the SQL definition of the query.

When PRODUCT_SALES is accessed, the database executes the underlying JOIN, WHERE, GROUP BY, and calculations again. Therefore, with 200 million rows, the query can still take a long time

(c) Rewrite (a) as a materialized view and describe when and how it should be refreshed so the team gets yesterday's figures in a few seconds.

407	SELECT
408	P.Prod_Name,
409	P.Category,
410	SUM(OI.Qty) AS Total_Qty,
411	SUM(OI.Qty * P.Price) AS Total_Revenue
412	FROM PRODUCT P
413	JOIN ORDER_ITEM OI
414	ON P.Prod_ID = OI.Prod_ID
415	JOIN ORDERS3 O
416	ON OI.Order_ID = O.Order_ID

Result Grid			Filter Rows:	Export:	W
	Prod_Name	Category	Total_Qty	Total_Revenue	
▶	Laptop	Electronics	3	150000.00	
	Mouse	Electronics	8	8000.00	
	Chair	Furniture	2	10000.00	
	Keyboard	Electronics	2	4000.00	

(d) If the column Category is later renamed to Prod_Category in the PRODUCT table, what happens to the view, and what must the DBA do?

If Category is renamed to Prod_Category, the existing view may become invalid because it still refers to the old column name Category.

The DBA must update or recreate the view so that it uses Prod_Category instead of Category. The same change should also be made to the stored PRODUCT_SALES_MV table/query if it is being refreshed.

Part: 4(Indexing):

Question 1: A movie-ticket website stores BOOKING(Booking_ID, Cust_ID, Show_ID, Booking_Date, Seat_No, Amount) with about 50 million rows.

Customers complain that the “My Bookings” page is slow. The page runs:
 SELECT Show_ID, Booking_Date, Seat_No, Amount FROM BOOKING
 WHERE Cust_ID = ? ORDER BY Booking_Date DESC.

(a) Propose the best single index for this query, giving the exact CREATE INDEX statement. Explain the column order you chose and why the ORDER BY benefits from it.

```

458 #1.a)
459 • CREATE INDEX idx_booking_cust_date
460 ON BOOKING (Cust_ID, Booking_Date DESC);
461 • SHOW INDEX FROM BOOKING;
462

```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality
booking	0	PRIMARY	1	Booking_ID	A	6
booking	1	idx_booking_cust_date	1	Cust_ID	A	4
booking	1	idx_booking_cust_date	2	Booking_Date	D	6

(b) Would an index on Seat_No (values 1 to 200) help queries such as WHERE Seat_No = 45? Justify your answer using the idea of selectivity.

```

464 #1.b):
465 • CREATE INDEX idx_booking_seat
466 ON BOOKING (Seat_No);
467 • SELECT *
468 FROM BOOKING
469 WHERE Seat_No = 45;

```

Booking_ID	Cust_ID	Show_ID	Booking_Date	Seat_No	Amount
1	101	501	2026-09-25 18:30:00	45	500.00
3	101	503	2026-09-27 19:00:00	45	550.00
6	104	505	2026-09-30 17:30:00	45	450.00
*	NULL	NULL	NULL	NULL	NULL

*This means the column has low selectivity. For example, Seat_No = 45 could match a large number of rows. When many rows match a condition, using the index may not provide much performance benefit, and the database optimizer may prefer a table scan.

(c) The analytics team also runs SELECT Show_ID, COUNT(*) FROM BOOKING WHERE Booking_Date BETWEEN ? AND ? GROUP BY Show_ID. Can the index from (a) serve this query? If not, what index would you add?

No, the index from part (a) cannot efficiently serve this query because it is ordered first by Cust_ID and then by Booking_Date. The query does not filter on Cust_ID,

so the leftmost-prefix rule prevents efficient use of Booking_Date for the range condition. A separate composite index on (Booking_Date, Show_ID) should be added. Booking_Date is placed first because it is used in the WHERE range condition, while Show_ID helps with the grouping operation

(d) Which command would you use to confirm that your index is actually used, and what would you look for in its output?

```
171 #1.d):
172 • EXPLAIN
173 SELECT Show_ID, Booking_Date, Seat_No, Amount
174 FROM BOOKING
175 WHERE Cust_ID = 101
176 ORDER BY Booking_Date DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

id	select_type	table	partitions	type	possible_keys	key	key_len
1	SIMPLE	BOOKING	NULL	ref	idx_booking_cust_date	idx_booking_cust_date	5

Question 2: A warehouse system's INVENTORY_LOG table receives about 10,000 INSERTs per minute from barcode scanners. A previous DBA created 12 indexes on it, and now both inserts and reports are slow. The three most common queries are: (1) WHERE Item_Code = ?, (2) WHERE Log_Time BETWEEN ? AND ?, and (3) WHERE Warehouse_ID = ? AND Item_Code = ?.

(a) Explain, in terms of what happens on every INSERT, why having 12 indexes slows down data loading.

Having 12 indexes makes INSERT operations slower because whenever a new record is inserted, the database has to update the table as well as all the indexes related to it. So, for every new row, the database has to maintain 12 index entries. Since the table receives around 10,000 inserts per minute, this extra work takes more time and resources.

Therefore, keeping too many unnecessary indexes can make data insertion slower.

(b) Design a minimal set of indexes (at most three) that supports all three queries. Show how one composite index can serve two of them using the leftmost-prefix rule.

```

497 #2 b):
498 • CREATE INDEX idx_item_code
499 ON INVENTORY_LOG(Item_Code);
500
501 • CREATE INDEX idx_log_time
502 ON INVENTORY_LOG(Log_Time);
503
504 • CREATE INDEX idx_warehouse_item
505 ON INVENTORY_LOG(Warehouse_ID, Item_Code);
506 • SHOW INDEX FROM INVENTORY_LOG;

```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinali
inventory_log	0	PRIMARY	1	Log_ID	A	6
inventory_log	1	idx_item_code	1	Item_Code	A	4
inventory_log	1	idx_warehouse_item	1	Warehouse_ID	A	3
inventory_log	1	idx_warehouse_item	2	Item_Code	A	4

(c) One index is a hash index on Log_Time. Explain why it cannot help query (2) and what type of index should replace it.

```

508 #2 c):
509 • CREATE INDEX idx_log_time_btree
510 ON INVENTORY_LOG(Log_Time);
511 • SHOW INDEX FROM INVENTORY_LOG;
512

```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinali
inventory_log	0	PRIMARY	1	Log_ID	A	6
inventory_log	1	idx_item_code	1	Item_Code	A	4
inventory_log	1	idx_warehouse_item	1	Warehouse_ID	A	3
inventory_log	1	idx_warehouse_item	2	Item_Code	A	4
inventory_log	1	idx_log_time_btree	1	Log_Time	A	6

(d) Suggest a way to keep the barcode inserts fast during the day while still allowing heavy reporting queries at night (think about materialized views or summary tables from Section 4).

Use a summary table or materialized view for reporting. During the day, keep the INVENTORY_LOG table focused on fast barcode inserts. At night, refresh/update the summary table and run heavy reports on it instead of the main table.

This keeps daytime inserts fast and allows efficient reporting at night.

Part: 5(Window Functions):

Question 1: A company records sales in SALES(Sale_ID, Region, Salesperson, Sale_Date, Amount).

(a) Rank the salespersons within each region by their total sales (highest first). Decide whether RANK, DENSE_RANK or ROW_NUMBER is most appropriate for a “top performer” award when two people have equal totals, and justify your choice.

```
536 • SELECT Region, Salesperson, Total_Sales,
537       RANK() OVER (
538         PARTITION BY Region
539         ORDER BY Total_Sales DESC
540       ) AS Sales_Rank
541 FROM (
542   SELECT Region, Salesperson, SUM(Amount) AS Total_Sales
543   FROM SALES
544   GROUP BY Region, Salesperson
545 ) AS T;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Region	Salesperson	Total_Sales	Sales_Rank
▶	North	Aman	11000.00	1
	North	Ravi	11000.00	1
	South	Neha	11000.00	1
	South	Karan	10000.00	2

(b) Produce a month-wise running total of sales for each region (hint: group to monthly totals first in a subquery or CTE, then apply the window).

```
548 • SELECT Region, Sale_Month, Monthly_Sales,
549       SUM(Monthly_Sales) OVER (
550         PARTITION BY Region
551         ORDER BY Sale_Month
552       ) AS Running_Total
553 FROM (
554   SELECT Region,
555          DATE_FORMAT(Sale_Date, '%Y-%m') AS Sale_Month,
556          SUM(Amount) AS Monthly_Sales
557   FROM SALES
558   GROUP BY Region, DATE_FORMAT(Sale_Date, '%Y-%m')
559 ) AS T
560 ORDER BY Region, Sale_Month;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Region	Sale_Month	Monthly_Sales	Running_Total
▶	North	2026-01	12000.00	12000.00
	North	2026-02	10000.00	22000.00
	South	2026-01	16000.00	16000.00
	South	2026-02	5000.00	21000.00

(c) For every sale, show the amount of the same salesperson's previous sale and the percentage change. What value appears for a salesperson's first sale, and how would you display 0 instead?

```

562 #1.c):
563 • SELECT Sale_ID, Salesperson, Sale_Date, Amount,
564
565 COALESCE(
566     LAG(Amount) OVER (
567         PARTITION BY Salesperson
568         ORDER BY Sale_Date
569     ), 0
570 ) AS Previous_Amount,
571

```

Result Grid						
	Sale_ID	Salesperson	Sale_Date	Amount	Previous_Amount	Percentage_Change
1	1	Aman	2026-01-10	5000.00	0.00	0.00
3	3	Aman	2026-02-05	6000.00	5000.00	20.00
6	6	Karan	2026-01-20	8000.00	0.00	0.00
8	8	Karan	2026-02-15	2000.00	8000.00	-75.00
5	5	Neha	2026-01-08	8000.00	0.00	0.00
7	7	Neha	2026-02-10	3000.00	8000.00	-62.50
2	2	Ravi	2026-01-15	7000.00	0.00	0.00
4	4	Ravi	2026-02-12	4000.00	7000.00	-42.86

(d) List the top 3 salespersons of every region. Explain why the ranking must be computed in an inner query before it can be filtered.

```

580 • SELECT *
581 FROM (
582     SELECT Region, Salesperson, Total_Sales,
583         RANK() OVER (
584             PARTITION BY Region
585             ORDER BY Total_Sales DESC
586         ) AS Sales_Rank
587     FROM (
588         SELECT Region, Salesperson, SUM(Amount) AS Total_Sales
589         FROM SALES
590         GROUP BY Region, Salesperson

```

Result Grid				
	Region	Salesperson	Total_Sales	Sales_Rank
1	North	Aman	11000.00	1
2	North	Ravi	11000.00	1
3	South	Neha	11000.00	1
4	South	Karan	10000.00	2

Question 2: A college stores STUDENT_MARKS(Roll_No, Student_Name, Subject, Marks).

- (a) For each subject, show every student with their marks, the subject's highest marks (using FIRST_VALUE with the correct frame) and the difference between the student's marks and the subject average, without losing any student rows.

```

626 FIRST_VALUE(Marks) OVER (
627     PARTITION BY Subject
628     ORDER BY Marks DESC
629     ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
630 ) AS Highest_Marks,
631
632 ROUND(

```

Roll_No	Student_Name	Subject	Marks	Highest_Marks	Difference_From_Average
2	Ravi	DBMS	92	92	5.25
4	Hardik	DBMS	92	92	5.25
1	Madhav	DBMS	85	92	-1.75
3	Keshav	DBMS	78	92	-8.75
1	Madhav	DSA	90	90	6.25
3	Keshav	DSA	90	90	6.25
2	Ravi	DSA	80	90	-3.75
4	Hardik	DSA	75	90	-8.75
1	Madhav	Java	88	88	6.00
2	Ravi	Java	88	88	6.00
4	Hardik	Java	82	88	0.00
3	Keshav	Java	70	88	-12.00

- (b) Assign each student a DENSE_RANK within each subject and then find students who are ranked 1 in more than one subject.

```

641 #2.b):
642 • SELECT Roll_No, Student_Name, Subject, Marks,
643        DENSE_RANK() OVER (
644            PARTITION BY Subject
645            ORDER BY Marks DESC
646        ) AS Subject_Rank
647 FROM STUDENT_MARKS;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Roll_No	Student_Name	Subject	Marks	Subject_Rank
▶	2	Ravi	DBMS	92	1
	4	Hardik	DBMS	92	1
	1	Madhav	DBMS	85	2
	3	Keshav	DBMS	78	3
	1	Madhav	DSA	90	1
	3	Keshav	DSA	90	1
	2	Ravi	DSA	80	2
	4	Hardik	DSA	75	3
	1	Madhav	Java	88	1
	2	Ravi	Java	88	1
	4	Hardik	Java	82	2
	3	Keshav	Java	70	3

```

649 • SELECT Student_Name
650 FROM (
651     SELECT Student_Name, Subject,
652            DENSE_RANK() OVER (
653                PARTITION BY Subject
654                ORDER BY Marks DESC
655            ) AS Subject_Rank
656     FROM STUDENT_MARKS
657 ) AS Ranked
658 WHERE Subject_Rank = 1
659 GROUP BY Student_Name
660 HAVING COUNT(*) > 1;
661

```

Result Grid | Filter Rows: | Export:

	Student_Name
▶	Ravi
	Madhav

(c) Divide the students of each subject into 4 performance bands using NTILE(4) and count how many students are in band 1 for the subject 'DBMS'.

```

662 #2.c):
663 • SELECT Student_Name, Subject, Marks, Band
664 FROM (
665     SELECT Student_Name, Subject, Marks,
666            NTILE(4) OVER (
667                PARTITION BY Subject
668                ORDER BY Marks DESC, Roll_No
669            ) AS Band
670     FROM STUDENT_MARKS
671 ) AS T
672 WHERE Subject = 'DBMS';
673

```

Student_Name	Subject	Marks	Band
Ravi	DBMS	92	1
Hardik	DBMS	92	2
Madhav	DBMS	85	3
Keshav	DBMS	78	4

(d) Explain in your own words what would change in the output of (a) if GROUP BY Subject were used instead of a window function, and why WHERE cannot be used to filter on a window function result.

GROUP BY Subject combines students and gives one row per subject, so individual student details are lost. A window function keeps every student row while calculating the subject-level values.

WHERE cannot filter a window function directly because window functions are calculated after WHERE. So we use a subquery or CTE first.

Part:6(CTE Report):

Question 1: An online shop organizes products in a category tree stored as CATEGORY(Cat_ID, Cat_Name, Parent_Cat_ID). For example: Electronics → Mobiles → Smartphones → Android Phones; Electronics → Laptops.

(a) Write a recursive CTE that lists all sub-categories of 'Electronics' at every depth, showing the depth number and the full path such as 'Electronics > Mobiles > Smartphones'.

694	•	WITH RECURSIVE category_tree AS (
695		-- Anchor member
696		SELECT
697		Cat_ID,
698		Cat_Name,
699		Parent_Cat_ID,
700		AS Depth,
701		CAST(Cat_Name AS CHAR(500)) AS Full_Path
702		FROM CATEGORY
703		WHERE Cat_Name = 'Electronics'
704		UNION ALL
705		
706		
707		

Cat_ID	Cat_Name	Depth	Full_Path
1	Electronics	0	Electronics
2	Mobiles	1	Electronics > Mobiles
5	Laptops	1	Electronics > Laptops
3	Smartphones	2	Electronics > Mobiles > Smartphones
4	Android Phones	3	Electronics > Mobiles > Smartphones > Android...

(b) Identify the anchor member and the recursive member in your query and explain how the database knows when to stop.

Anchor Member:

The anchor member starts the recursive CTE with the root category Electronics.

Recursive Member:

The recursive member finds the sub-categories by matching Parent_Cat_ID of a category with the Cat_ID of the current category.

Stopping Condition:

The recursion stops automatically when no more child categories are found.

(c) Modify the query so that it also counts, for each category, the number of products in PRODUCT(Prod_ID, Prod_Name, Cat_ID) that belong to that category or any of its sub-categories.

```

760 SELECT
761     c.Cat_ID,
762     c.Cat_Name,
763     COUNT(p.Prod_ID) AS Product_Count
764 FROM CATEGORY c
765 LEFT JOIN category_tree ct
766     ON c.Cat_ID = ct.Main_Cat_ID
767 LEFT JOIN PRODUCTS p
768     ON p.Cat_ID = ct.Sub_Cat_ID
769 GROUP BY c.Cat_ID, c.Cat_Name
770 ORDER BY c.Cat_ID;

```

Result Grid			
Filter Rows:			
Export: 			
	Cat_ID	Cat_Name	Product_Count
▶	1	Electronics	6
	2	Mobiles	4
	3	Smartphones	3
	4	Android Phones	2
	5	Laptops	2

(d) What would happen if, by a data-entry mistake, 'Smartphones' had 'Android Phones' as its parent (a cycle)? How does your database protect against this?

A cycle causes the recursive CTE to repeatedly visit the same categories. MySQL limits recursive CTE execution using a maximum recursion depth, preventing unlimited recursion. However, cycles should be prevented in the data itself.

Question 2: A bank stores TRANSACTIONS(Txn_ID, Acc_No, Txn_Date, Txn_Type, Amount) where Txn_Type is 'D' for deposit or 'W' for withdrawal.

(a) Using two CTEs, compute the monthly total deposits and the monthly total withdrawals for each account.

```

817 SELECT
818     d.Acc_No,
819     d.Month,
820     d.Total_Deposits,
821     w.Total-Withdrawals
822 FROM deposits d
823 LEFT JOIN withdrawals w
824     ON d.Acc_No = w.Acc_No
825     AND d.Month = w.Month;

```

Result Grid				
Filter Rows:				
Export: 				
	Acc_No	Month	Total_Deposits	Total-Withdrawals
	101	2026-01	10000.00	3000.00
	101	2026-02	5000.00	2000.00
	102	2026-01	8000.00	NULL
	102	2026-03	6000.00	1000.00

(b) Add a third CTE that joins the first two and calculates the net change (deposits – withdrawals) per account per month. Handle months where an account has deposits but no withdrawals (or vice versa) so that they are not lost.

```

865      AS Total_Deposits,
866      w.Total_Withdrawals,
867      - w.Total_Withdrawals AS Net_Change
868  FROM withdrawals w
869  LEFT JOIN deposits d
870      ON w.Acc_No = d.Acc_No
871      AND w.Month = d.Month
872  WHERE d.Acc_No IS NULL
873  )
874
875  SELECT *
876  FROM net_change
877  ORDER BY Acc_No, Month;

```

Acc_No	Month	Total_Deposits	Total_Withdrawals	Net_Change
101	2026-01	10000.00	3000.00	7000.00
101	2026-02	5000.00	2000.00	3000.00
101	2026-03	0.00	4000.00	-4000.00
102	2026-01	8000.00	0.00	8000.00
102	2026-02	0.00	3000.00	-3000.00

(c) In the main query, flag the months where withdrawals exceeded deposits with the label 'OVERDRAWN', and show a running balance per account using a window function over the net change.

```

931      Total_Withdrawals,
932      Net_Change,
933      CASE
934          WHEN Total_Withdrawals > Total_Deposits
935          THEN 'OVERDRAWN'
936          ELSE 'OK'
937      END AS Status,
938      SUM(Net_Change) OVER (
939          PARTITION BY Acc_No
940          ORDER BY Month
941      ) AS Running_Balance
942  FROM net_change

```

Acc_No	Month	Total_Deposits	Total_Withdrawals	Net_Change	Status	Running_Bal
101	2026-02	5000.00	2000.00	3000.00	OK	10000.00
101	2026-03	0.00	4000.00	-4000.00	OVERDRAWN	6000.00
102	2026-01	8000.00	0.00	8000.00	OK	8000.00
102	2026-02	0.00	3000.00	-3000.00	OVERDRAWN	5000.00
102	2026-03	6000.00	1000.00	5000.00	OK	10000.00

(d) Rewrite the CTE from (a) as a derived table (subquery in FROM) and write two or three sentences comparing readability and reusability of the two versions.

```
948     Acc_No,  
949     Month,  
950     Total_Deposits  
951 FROM (  
952     SELECT  
953         Acc_No,  
954         DATE_FORMAT(Txn_Date, '%Y-%m') AS Month,  
955         SUM(Amount) AS Total_Deposits  
956     FROM TRANSACTIONS  
957     WHERE Txn_Type = 'D'  
958     GROUP BY Acc_No, DATE_FORMAT(Txn_Date, '%Y-%m')  
959 ) AS deposits;
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Conte

	Acc_No	Month	Total_Deposits
	101	2026-01	10000.00
	101	2026-02	5000.00
	102	2026-01	8000.00
	102	2026-03	6000.00